SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1702**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I Vijiya Priya S.A (192524417) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

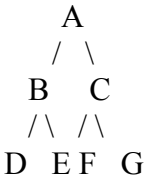
Vijiya Priya S.A
192524417

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



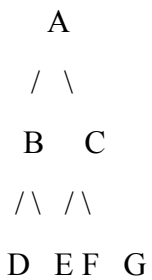
Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.
Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

Background: Uninformed (Blind) Search

All three questions use **uninformed search**: the algorithm has no knowledge about how close a state is to the goal beyond the problem definition itself (initial state, successor function, goal test and step cost). A search algorithm builds a *search tree* from the initial state; the set of generated but not yet expanded nodes is the **frontier** (open list), and the data structure used for the frontier decides the strategy. Every dry run in this document therefore tracks (i) the frontier before and after each iteration, (ii) the node selected for expansion, and (iii) the nodes generated.

Strategy	Frontier structure	Node selected for expansion	Used in
Breadth-First Search	FIFO queue	Shallowest node (oldest in queue)	Q1 and Q3
Depth-First Search	LIFO stack	Deepest node (newest on stack)	Q2
Uniform-Cost Search	Priority queue ordered by g(n)	Node with the least path cost	Related (weighted graphs)

The strategies are compared using four standard measures: **completeness** (does it find a solution when one exists?), **optimality** (is the solution found the cheapest?), **time complexity** (number of nodes generated) and **space complexity** (maximum nodes stored). With branching factor b, solution depth d and maximum depth m, BFS has time and space O(b^d), whereas DFS has time O(b^m) and space only O(b·m).

Question 1: Breadth-First Search (BFS) on a Binary Tree

> *Problem: Starting from node A, perform Breadth-First Search on the graph with edges A–B, A–C, B–D, B–E, C–F and C–G. Complete the iteration table, write the traversal order and show the queue contents after each iteration.*

1.1 Understanding the problem

The graph is a complete binary tree of depth 2 with seven nodes. A is the **root** (level 0), B and C are at level 1, and D, E, F, G are the **leaves** at level 2. Because the graph is a tree, there are no cycles, so a node can only be reached through its parent and no node is ever generated twice. When the question says a node is "visited", it means the node has been **removed from**

the front of the queue and processed. Children are always added to the queue in **left-to-right order**.

1.2 BFS strategy and algorithm

BFS is an **uninformed** search that expands the shallowest unexpanded node first. It uses a **FIFO queue** as its frontier: nodes are inserted at the rear and removed from the front, so all nodes of depth d are processed before any node of depth d+1.

```
BFS(start):
 queue <- [start];  visited <- {start}
 while queue is not empty:
    node <- DEQUEUE(queue)          # visit the front node
    print node
    for each child of node (left to right):
       if child not in visited:
          ENQUEUE(queue, child);  visited <- visited + {child}
```

1.3 Dry run – iteration table

The "Queue" column shows the queue **at the start of the iteration** (before the front node is removed); the "Visited Node" column shows the node removed and processed in that iteration.

Iteration	Queue (front → rear)	Visited Node
1	[A]	A
2	[B, C]	B
3	[C, D, E]	C
4	[D, E, F, G]	D
5	[E, F, G]	E
6	[F, G]	F
7	[G]	G

1.4 Queue contents after each iteration

Iteration	Node removed	Children added (rear)	Queue after the iteration
1	A	B, C	[B, C]
2	B	D, E	[C, D, E]
3	C	F, G	[D, E, F, G]
4	D	none (leaf)	[E, F, G]
5	E	none (leaf)	[F, G]
6	F	none (leaf)	[G]
7	G	none (leaf)	[ ] (empty – search ends)

1.5 BFS traversal order

BFS traversal order: A → B → C → D → E → F → G (level order: level 0 = A; level 1 = B, C; level 2 = D, E, F, G).

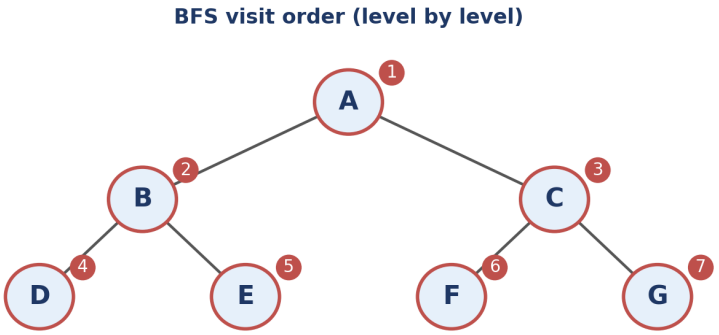


Figure 1: BFS visit order shown on the tree (numbers give the order of visit).

1.6 Observations

5. The queue never holds more than **four** nodes (maximum at iteration 4); this peak equals the width of the widest level, which is why BFS needs O(b^d) memory.
6. Every node is enqueued exactly once and dequeued exactly once, so the running time is O(V + E) = O(7 + 6), i.e., 7 iterations for 7 nodes.
7. BFS finds the **shallowest** goal. For example, G would be found at depth 2 with path A → C → G.

1.7 Path reconstruction using parent pointers

To return a path (not just an order) BFS stores a **parent pointer** for each node when it is first generated. Following the pointers from any node back to A gives the shortest path to that node, which for BFS is also the fewest-edge path:

Node	Parent	Depth	Shortest path from A
A	–	0	A
B	A	1	A → B
C	A	1	A → C
D	B	2	A → B → D
E	B	2	A → B → E
F	C	2	A → C → F
G	C	2	A → C → G

Complexity check for this tree: with branching factor b = 2 and depth d = 2, BFS generates 1 + 2 + 4 = 7 nodes, matching the 7 iterations performed; the maximum queue length is 4 = b^d, which confirms the O(b^d) memory requirement.

Question 2: Depth-First Search (DFS) on the Same Graph

> *Problem: Using the same graph, perform Depth-First Search starting from A and complete the table.*

2.1 DFS strategy and algorithm

DFS is an uninformed search that always expands the **deepest** unexpanded node. Its frontier is a **LIFO stack** (or, equivalently, the recursion call stack): the most recently generated node is processed first. The search follows one branch all the way down to a leaf, then **backtracks** to the nearest ancestor that still has unexplored children. To visit children in left-to-right order with an explicit stack, the children are pushed in **reverse order** (right child first, left child last), so that the left child is on top.

```
DFS(start):
  stack <- [start];  visited <- {}
  while stack is not empty:
     node <- POP(stack)            # take the top node
```

```
if node not in visited:
    print node;  visited <- visited + {node}
    for each child of node (right to left):
        PUSH(stack, child)
```

2.2 Dry run – iteration table (stack shown top → bottom)

Iteration	Stack before (top → bottom)	Visited Node	Stack after (children pushed)
1	[A]	A	[B, C] (push C, then B)
2	[B, C]	B	[D, E, C] (push E, then D)
3	[D, E, C]	D	[E, C] (leaf – nothing pushed)
4	[E, C]	E	[C] (leaf; backtrack to C)
5	[C]	C	[F, G] (push G, then F)
6	[F, G]	F	[G] (leaf)
7	[G]	G	[ ] (empty – search ends)

2.3 DFS traversal order

DFS traversal order: A → B → D → E → C → F → G. The search goes down the left branch first (A, B, D); after reaching the leaf D it backtracks to B and visits E; since B has no more children it backtracks to A and then explores the right branch (C, F, G).

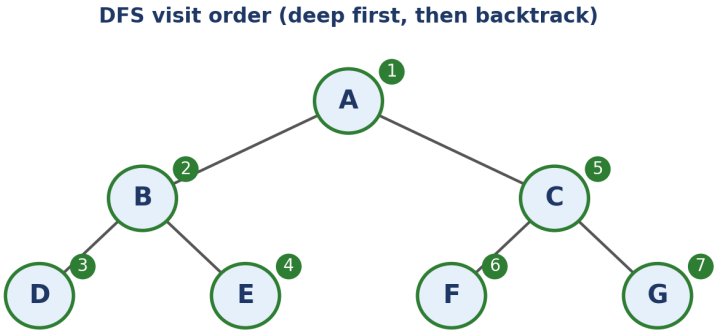


Figure 2: DFS visit order shown on the tree.

2.4 BFS vs DFS on this graph

Property	BFS	DFS
Data structure	Queue (FIFO)	Stack (LIFO) / recursion
Order on this tree	A, B, C, D, E, F, G	A, B, D, E, C, F, G
Maximum frontier size	4 nodes (level 2)	3 nodes (e.g., D, E, C)
Time complexity	O(b^d)	O(b^m), m = maximum depth
Space complexity	O(b^d) – large	O(b·m) – small
Completeness	Complete (finite branching)	Complete only in finite, acyclic spaces (or with a visited list)
Optimality (unit cost)	Optimal – finds shallowest goal	Not optimal – may reach a deeper goal first

For this small tree both algorithms visit all seven nodes in seven iterations; they differ only in order. DFS would find leaf G last, whereas BFS finds all level-2 nodes before going deeper, so for a search tree with a goal at shallow depth BFS is preferable, whereas for very deep trees with limited memory DFS is preferable.

2.5 Recursive view of DFS (call-stack trace)

The same traversal can be written recursively: DFS(node) prints the node and then calls DFS on each child from left to right. The system call stack plays the role of the explicit stack, and returning from a call is the **backtracking** step.

Step	Action	Call stack after the action (bottom → top)
1	Call DFS(A), visit A	A
2	Call DFS(B), visit B	A, B
3	Call DFS(D), visit D	A, B, D
4	D has no children – return (backtrack to B)	A, B
5	Call DFS(E), visit E	A, B, E
6	E returns, then B returns (backtrack to A)	A

Step	Action	Call stack after the action (bottom → top)
7	Call DFS(C), visit C	A, C
8	Call DFS(F), visit F; F returns	A, C
9	Call DFS(G), visit G; G, C and A return	(empty) – done

The recursion depth never exceeds 3 frames (A, B, D) – the tree height plus one – which illustrates the O(b·m) space advantage of DFS. Common variants that fix the weaknesses of plain DFS are **Depth-Limited Search** (stop at a fixed depth limit l to avoid infinite branches) and **Iterative Deepening DFS**, which repeats depth-limited search with l = 0, 1, 2, … to combine DFS's small memory use with BFS's completeness and optimality.

Question 3: Breadth-First Search on the 8-Puzzle

> *Problem: Initial state 1 3 6 / 5 _ 2 / 4 7 8 and goal state 1 2 3 / 4 5 6 / 7 8 _ (the blank is shown as 0). Trace BFS until the goal state is reached, fill in the table, and answer the questions.*

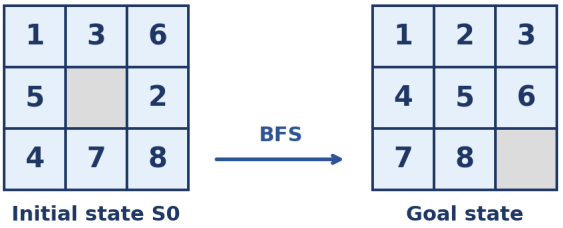


Figure 3: Initial and goal states of the 8-puzzle.

3.1 Problem formulation

Component	Definition
State	A 3×3 arrangement of tiles 1–8 and the blank (0). Written compactly as row1/row2/row3, e.g., the initial state is 136/502/478.
Initial state S0	136/502/478 (blank in the centre).
Goal state	123/456/780.
Operators	Move the blank Up, Down, Left or Right (when the move stays inside the board). Applied in this fixed order: **Up, Down, Left, Right**.
Step cost	1 per move (unweighted), so path cost = number of moves.
Duplicate check	A state already generated is **not** added again (explored/visited set), so the move that undoes the previous move is discarded.

Solvability check: counting inversions (pairs of tiles in the wrong order, ignoring the blank) in the reading order 1,3,6,5,2,4,7,8 gives 6 inversions (3>2, 6>5, 6>2, 6>4, 5>2, 5>4), which is **even**. The goal has 0 inversions. For a 3×3 board, a state is solvable if its inversion count has the same parity as the goal's, so the puzzle is **solvable**.

3.2 BFS dry run – first five iterations (as requested in the table)

States are named S0, S1, S2, … in the order they are generated. Each row gives the state being expanded, the queue before expansion (the current state is at the front), and the queue after expansion (after removing the current state and appending its new children).

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	S0 = 136/502/478	S0	S1, S2, S3, S4
2	S1 = 106/532/478	S1, S2, S3, S4	S2, S3, S4, S5, S6
3	S2 = 136/572/408	S2, S3, S4, S5, S6	S3, S4, S5, S6, S7, S8
4	S3 = 136/052/478	S3, S4, S5, S6, S7, S8	S4, S5, S6, S7, S8, S9, S10
5	S4 = 136/520/478	S4, S5, S6, S7, S8, S9, S10	S5, S6, S7, S8, S9, S10, S11, S12

Key to the state labels and how each was generated:

Label	State	Generated from	Operator (blank moves)
S1	106/532/478	S0	Up
S2	136/572/408	S0	Down
S3	136/052/478	S0	Left
S4	136/520/478	S0	Right
S5, S6	016/532/478, 160/532/478	S1	Left, Right
S7, S8	136/572/048, 136/572/480	S2	Left, Right
S9, S10	036/152/478, 136/452/078	S3	Up, Down
S11, S12	130/526/478, 136/528/470	S4	Up, Down

Note that S0 has four children because the blank is in the centre. S1–S4 each have only two **new** children because the move that returns to S0 is discarded as a duplicate (for example, S1

could move the blank Down to return to S0). After five iterations, **13 states** (S0–S12) have been generated and 5 have been expanded.

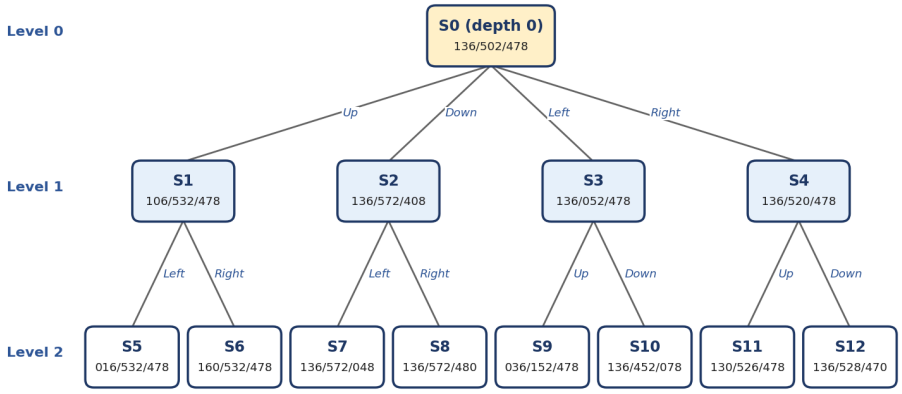


Figure 4: Search tree of the 8-puzzle up to depth 2 (labels S0–S12 as used in the tables).

3.3 Continuing BFS until the goal

The goal is at depth 8, so the table cannot end after five rows: BFS must expand **every node of depth 0 to 7** and then part of depth 8 before the goal reaches the front of the queue. The exact numbers of states at each depth (computed by running BFS with the operator order and duplicate check above) are:

Depth	States at this depth	Expanded up to end of depth	Iterations covering this depth	Remark
0	1	1	1	S0
1	4	5	2 – 5	S1–S4
2	8	13	6 – 13	
3	8	21	14 – 21	
4	16	37	22 – 37	
5	32	69	38 – 69	
6	60	129	70 – 129	
7	72	201	130 – 201	

Depth	States at this depth	Expanded up to end of depth	Iterations covering this depth	Remark
8	136	311 (stop)	202 – 311	Goal is the 110th node at depth 8 → **iteration 311**

3.4 Solution found

The goal 123/456/780 is removed from the queue and recognised as the goal in **iteration 311**. Following the parent pointers back to S0 gives the solution path below. The ancestor of the goal that is expanded at each depth, and the iteration in which this happens, are also shown:

Depth	Move (blank)	State	Expanded in iteration	Comment
0	–	136/502/478	1	Initial state
1	Right	136/520/478	5	This is S4
2	Up	130/526/478	12	This is S11
3	Left	103/526/478	20	
4	Down	123/506/478	34	Tiles 1, 2, 3 in place
5	Left	123/056/478	63	
6	Down	123/456/078	119	Tiles 1–6 in place
7	Right	123/456/708	187	
8	Right	123/456/780	311	**Goal reached**

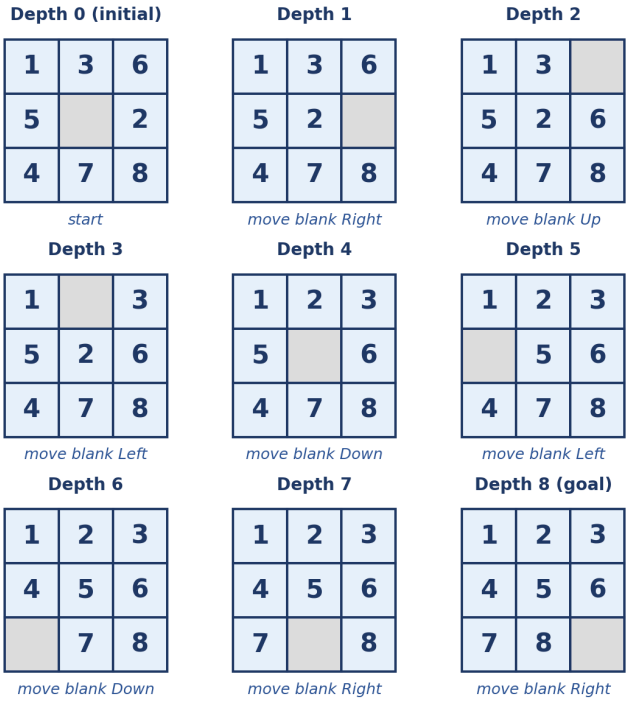


Figure 5: Complete solution path from the initial state to the goal (8 moves).

3.5 Answers to the questions

(a) Which node is expanded in each iteration? BFS expands nodes strictly in the order in which they were generated (FIFO). For the first five iterations the nodes expanded are S0, S1, S2, S3 and S4, i.e., the root first and then its four children in the order Up, Down, Left, Right. Iterations 6–13 expand the depth-2 states (S5–S12), and so on level by level until iteration 311, in which the goal state is selected for expansion and the search stops.

(b) How many states are generated in total? When the goal is chosen for expansion (goal test on removal from the queue) a total of **500 states are generated** (including the initial state): all 337 states of depth 0–8 plus 163 depth-9 states created while expanding depth-8 nodes. **311 states are expanded** and 189 remain waiting in the queue. If the goal test is applied when a state is *generated*, the search stops earlier, at iteration 187, with **311 states generated**. Either way the number of states is tiny compared with the 9!/2 = 181,440 reachable states, but the growth rate is roughly ×2 per level.

(c) Complete solution path. The blank moves **Right, Up, Left, Down, Left, Down, Right, Right** (8 moves):

136/502/478 → 136/520/478 → 130/526/478 → 103/526/478 → 123/506/478 → 123/056/478 → 123/456/078 → 123/456/708 → **123/456/780 (goal)**

The solution cost is 8 and the depth of the goal is 8.

(d) Why does BFS always find the shortest solution in an unweighted 8-puzzle? In the 8-puzzle every move costs 1, so the path cost of a node equals its depth. BFS expands nodes in non-decreasing order of depth: no node of depth d+1 is expanded until every node of depth d has been expanded. Therefore, when the goal is first removed from the queue, it cannot have been reachable at a shallower depth (otherwise that shallower copy would have been generated and dequeued earlier). Hence the first solution found is the one with the **fewest moves**. BFS is also **complete**, because the branching factor is finite (at most 4) and the state space is finite. Its drawback is memory: it stores the whole frontier (189 states here, and growing exponentially with the solution depth), which is why informed methods such as A* with the Manhattan-distance heuristic are preferred for harder instances.

3.6 Growth of the frontier and the cost of BFS

The number of nodes per depth roughly doubles from depth 4 onwards (16, 32, 60, 72, 136) because each non-initial state has an average of about two new successors (the blank has 2, 3 or 4 possible moves and one of them undoes the previous move). The frontier grew to about **190 states** at its peak, although the solution is only 8 moves deep. This exponential growth illustrates that BFS is time- and memory-limited: a solution at depth 20 would require millions of states.

Measure	Value in this run	Meaning
Solution depth d	8	Fewest possible moves
States expanded	311	Iterations performed
States generated	500	Total nodes created (goal test on expansion)
Peak frontier size	about 190	Memory needed by BFS
Branching factor b	2 – 4 (average ≈ 2)	Controls the exponential growth

3.7 Comparison with an informed search (A*)

BFS ignores how close a state is to the goal. An informed search such as **A*** uses f(n) = g(n) + h(n); with the **Manhattan-distance** heuristic (sum of the horizontal and vertical distances of each tile from its goal position), the initial state has h = 1 (tile 3) + 1 (tile 6) + 1 (tile 5) + 2

(tile 2) + 1 (tile 4) + 1 (tile 7) + 1 (tile 8) = **8**, which is exactly the optimal solution length of 8. Because this heuristic is admissible and consistent, A* returns the same optimal 8-move solution, but in a run with ordinary tie-breaking it expanded only about **13 states** (against 311 for BFS), because every state on the optimal path keeps f(n) = 8, so the search is steered along that path. This shows why heuristics are central to CO2: BFS guarantees the shortest solution but pays a high price in time and memory, whereas A* keeps the guarantee at a fraction of the cost.

Summary of Results

Q	Problem	Result	Key point
1	BFS on the binary tree	A, B, C, D, E, F, G (7 iterations)	FIFO queue, level-order visit
2	DFS on the binary tree	A, B, D, E, C, F, G (7 iterations)	LIFO stack, backtracking
3	8-puzzle with BFS	Goal at depth 8, 311 expanded, 500 generated	Shortest path: Right, Up, Left, Down, Left, Down, Right, Right

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.